SEMINÁRIO DE ALGORITMOS

Maior Subsequência Comum

LCS · Longest Common Subsequence · Programação Dinâmica

B C B

Breno Ryan de Andrade Fernandes · UNISAL · Engenharia da Computação
João Vitor Ferreira Azevedo Pereira · UNISAL · Engenharia da Computação

1 · QUAL É O PROBLEMA?

Duas palavras. O que elas têm em comum?

X = A B C B

Y = B C A B

Qual é a **maior sequência de letras** que aparece nas duas, **na mesma ordem**, mesmo sem estarem lado a lado?

1 · QUAL É O PROBLEMA?

A resposta é BCB, com tamanho 3



As letras **B, C, B** aparecem nas duas palavras, nessa ordem. No Y, o A no meio é simplesmente pulado.

1 · QUAL É O PROBLEMA?

Subsequência não é substring

Subsequência

Pode **pular letras**, mas mantém a ordem.

ABCDE

“ACE” **é** subsequência de “ABCDE”.

Substring

Precisa ser **contínua**, sem pular nada.

BCDE

“ACE” **não é** substring de “ABCDE”; “BCD” é.

1 · QUAL É O PROBLEMA?

Onde isso aparece de verdade

diff e Git

Comparam duas versões de um arquivo e mostram quais linhas mudaram.

Bioinformática

Compara sequências de DNA para medir o quanto se parecem.

Plágio

Detecta trechos parecidos entre dois textos ou códigos.

2 · O QUE O ALGORITMO RESOLVE?

Entrada, saída e objetivo

ENTRADA

Duas sequências X e Y

X com m letras e Y com n letras.

```
X = "ABCB"
Y = "BCAB"
```

SAÍDA

O tamanho e a subsequência

Quantas letras tem a LCS e quais são elas.

```
tamanho = 3
LCS = "BCB"
```

OBJETIVO

A maior sequência comum

Que aparece nas duas, na mesma ordem, sem precisar ser contínua.

3 · QUAL É A ESTRATÉGIA?

E se testássemos todas as possibilidades?

A força bruta gera todas as subsequências de X e procura cada uma em Y. Uma palavra com m letras tem **2 elevado a m** subsequências.

1.073.741.824

subsequências para uma palavra com apenas 30 letras

3 · QUAL É A ESTRATÉGIA?

A solução: programação dinâmica



Subestrutura ótima

A resposta para as palavras inteiras é montada a partir das respostas para pedaços menores delas (os prefixos).



Subproblemas repetidos

Os mesmos pedaços aparecem várias vezes. Guardamos cada resultado em uma tabela e nunca calculamos de novo.



Por que não guloso?

Pegar a primeira letra igual pode estragar a resposta mais à frente, e o guloso nunca volta atrás.

4 · COMO O ALGORITMO FUNCIONA?

A ideia: uma tabela de respostas

`dp[i][j]` = tamanho da LCS entre as **i primeiras letras de X** e as **j primeiras letras de Y**

`dp[2][3] = 1`

LCS de “AB” e “BCA” tem 1 letra.

`dp[0][j] = 0`

Linha e coluna 0 são a palavra vazia: LCS sempre 0.

`dp[4][4] = 3`

O canto inferior direito é a resposta final.

4 · COMO O ALGORITMO FUNCIONA?

A regra que preenche cada célula



Letras iguais → diagonal + 1

```
dp[i][j] = dp[i-1][j-1] + 1
```



Letras diferentes → maior entre cima e esquerda

```
dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

4 · COMO O ALGORITMO FUNCIONA?

Pseudocódigo

```
LCS(X, Y):
    m ← tamanho de X;  n ← tamanho de Y
    criar tabela dp (m+1) × (n+1) com zeros
    para i de 1 até m:
        para j de 1 até n:
            se X[i] = Y[j]:
                dp[i][j] ← dp[i-1][j-1] + 1
            senão:
                dp[i][j] ← max(dp[i-1][j], dp[i][j-1])
    retornar dp[m][n]
```

ETAPA 1

Montar a tabela com zeros.

ETAPA 2

Preencher com a regra.

ETAPA 3

Voltar pela tabela para montar a palavra.

4 · TABELA PASSO A PASSO

Passo 0: a borda começa com zero

	∅	B	C	A	B
∅	0	0	0	0	0
A	0				
B	0				
C	0				
B	0				

Tabela de 5 × 5

X tem 4 letras e Y tem 4 letras, então a tabela tem (4+1) × (4+1) células.

A linha ∅ e a coluna ∅ representam a palavra vazia: a LCS com ela é sempre **0**.

Agora preenchemos linha por linha, da esquerda para a direita.

palavra vazia ainda não calculada

4 · TABELA PASSO A PASSO

Passo 1: linha A

	∅	B	C	A	B
∅	0	0	0	0	0
A	0	0	0	1	1
B	0				
C	0				
B	0				

A × A: letras iguais

`diagonal + 1 = 0 + 1 = 1`

A × B e A × C: diferentes → o maior entre cima e esquerda, que é 0.

A × B (última): diferentes → max(0, 1) = 1.

célula calculada de onde veio o valor

4 · TABELA PASSO A PASSO

Passo 2: linha B

	∅	B	C	A	B
∅	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	2
C	0				
B	0				

B × B (última): iguais

`diagonal + 1 = 1 + 1 = 2`

B × B (primeira): iguais → 0 + 1 = 1.

B × C e B × A: diferentes → repetem o maior vizinho, 1.

célula calculada de onde veio o valor

4 · TABELA PASSO A PASSO

Passo 3: linha C

	∅	B	C	A	B
∅	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	2
C	0	1	2	2	2
B	0				

C × A: letras diferentes

`max(cima 1, esquerda 2) = 2`

C × B: diferentes → max(1, 0) = 1.

C × C: iguais → diagonal 1 + 1 = 2.

C × B (última): diferentes → max(2, 2) = 2.

célula calculada de onde veio o valor

4 · TABELA PASSO A PASSO

Passo 4: última linha B

	∅	B	C	A	B
∅	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	2
C	0	1	2	2	2
B	0	1	2	2	**3**

B × B: iguais ⟶ resposta

`diagonal + 1 = 2 + 1 = 3`

B × B (primeira): iguais ⟶ 0 + 1 = 1.

B × C e B × A: diferentes ⟶ 2.

`dp[4][4] = 3`

A LCS de “ABCB” e “BCAB” tem 3 letras.

4 · TABELA PASSO A PASSO

Passo 5: voltar pela tabela

	∅	B	C	A	B
∅	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	2
C	0	1	2	2	2
B	0	1	2	2	3

Do canto até a borda

1. B = B → guarda **B**, anda na diagonal.

2. C ≠ A → vai para o vizinho maior (esquerda, 2).

3. C = C → guarda **C**, diagonal.

4. B = B → guarda **B**, chegou na borda.

Lido ao contrário: **B** **C** **B**

5 · TESTE

Executando com três conjuntos de dados

X	Y	LCS	Tamanho	Por que testar
ABCB	BCAB	BCB	3	O exemplo da apresentação
AGGTAB	GXTXAYB	GTAB	4	Exemplo clássico, maior
ABC	DEF	(vazia)	0	Caso limite: nada em comum

6 · COMPLEXIDADE

Quanto custa?

TEMPO

O(m × n)

Dois for aninhados e cada célula custa O(1). O backtracking soma só O(m + n).

ESPAÇO

O(m × n)

Por causa da tabela, que tem (m + 1) × (n + 1) células.

Com 30 letras em cada palavra:

≈ 900

operações com programação dinâmica

> 32 bilhões

com força bruta, O(2^m × n)

RESUMO

O que levar daqui

O problema

A maior sequência comum a duas palavras, na mesma ordem.

A estratégia

Programação dinâmica: uma tabela e uma regra de duas linhas.

O custo

O(m × n) em vez de exponencial.